PART IIB REVISION NOTES

4F14: Computer Systems

Qianshuo (Harry) Ye
April 2026

1 Introduction

Central processing unit (CPU): Components of the CPU are:

- *Control unit:* Fetches instructions from main memory and determines their type. Coordinates data flow around the CPU during instruction execution.
- *Arithmetic logic unit (ALU):* Performs simple arithmetic and logic functions on data within the CPU.
- *Registers:* Small, high speed memory used to store temporary results and control information.

Fetch-decode-execute cycle: Instructions are executed in the follow cycle:

1. *Fetch* next instruction from main memory.
2. Increment the PC to point to the next instruction.
3. *Decode* instruction.
4. If the instruction uses data from memory, calculate the relevant address and fetch the data.
5. *Execute* the instruction.
6. Store results.

Memory addressing:

- *Byte-addressable:* Each address referes to an 8-bit quantity (a byte).
- *Word:* A unit of data manipulated by the CPU (the size of the register). (4 bytes for a 32-bit processor)

$$\text{word address} = \left\lfloor \frac{\text{byte address}}{4} \right\rfloor$$

Byte ordering: Big endian (most significant byte stored at the lowest address) vs little endian (least significant byte stored at the lowest address).

Magnetic disks:

- Glass/ceramic platters coated in thin film magnetic media rotate at high speed.
- Data is read/written by electromagnetic heads that float above both surfaces of the platters.

- Each surface is divided into $\sim$ 280,000 *tracks*.
- The collection of tracks under the heads at a given time is called a *cylinder*.
- Each track is divided into $\sim$ 850-2600 *sectors*, which are the smallest unites that can be read/written.
- The time to read or write a sector of data to or from the disk is

  $$\text{access time} = \text{seek time} + \text{rotational latency} + \text{transfer time} + \text{controller overhead}$$

  - *Seek time:* Time taken to move the heads to the right cylinder.
  - *Rotational latency:* Time taken for the right sector to rotate under the head.
  - *Transfer time:* Time taken to read/write the data from/to the disk.
  - *Controller overhead:* Time taken by the disk controller to process the request.

Performance measurement: *Execution time* is the time fo perform a single job, while *throughput* is the total work done per unit time. The run-time can be broken down as:

$$\text{time} = \text{instructions} \times \frac{\text{clock cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{clock cycle}}$$

To improve performance for a fixed instruction set:

1. Reduce the number of instructions per program.
2. Reduce the number of clock cycles per instruction.
3. Reduce the clock cycle time.

FLOPS (floating point operations per second) is a common performance metric.

Amdahl's law: "Make the common case fast."

$$t_{\text{new}} = t_{\text{old}} \times \left[(1 - f) + \frac{f}{s} \right]$$

where f is the fraction of execution time affected by the improvement and $s = \frac{t_{\text{prog, old}}}{t_{\text{prog, new}}}$ is the speedup of the improved portion.

2 Instruction Set Architecture

Instruction: An *instruction = opcode + operands*.

- *Opcode:* Type of operation to perform.
- *Operands:* Data to be operated on.

Classification of ISAs:

- *Stack ISA*: Operands are taken from the stack.
- *Accumulator ISA:* One operand is taken from the accumulator.

- *General purpose register ISA:* All operands are explicitly declared, either registers or memory. The key parameters are: the *number of operands* per ALU instruction and the *maximum number of memory references* in ALU instructions.

MIPS ISA: A *load-store* architecture (only load and store instructions can access memory) with fixed-size 32-bit instructions and 32 general purpose registers. The operands can be registers or memory words.

Name	Example	Comments
32 registers	$0,$1 . . . $31	Very high speed. Data must be in registers to perform arithmetic.
2^{30} memory words	Memory[0] Memory[4] Memory[8] . . . Memory[2^{32}–4]	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential words differ by 4. Memory holds data and programs.

Assembly language: The symbolic representation of instructions, can be translated into *machine language* (executable code) by an assembler (also accepts *pseudo-instructions*).

R-format instruction:

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- op = operation of the instruction
- rs = first register source operand
- rt = second register source operand
- rd = register destination operand
- shamt = shift amount (for shift instructions)
- funct = selects variant of operation in op field

Examples:

- Add: `add $8,$17,$18` means $8 = $17 + $18. The machine language in decimal is [0, 17, 18, 8, 0, 32].
- Subtract: `sub $8,$17,$18` means $8 = $17 - $18.
- Move (pseudo): `move $8,$18` means $8 = $18. It is equivalent to `add $8,$0,$18`.
- Set on less than: `slt $1,$2,$3` means if $2 < $3, then $1 = 1; else $1 = 0.
- Jump register: `jr $8` means jump to the address in register $8 (PC = $8). The machine language in demcimal is [0, 8, 0, 0, 0, 8].

I-format instruction:

6 bits	5 bits	5 bits	16 bits
op	rs	rt	address

The *addressing modes* specifies how an instrunction accesses its operands:

- *Register:* operand is a register.

- *Immediate:* operand is a constant value.
- *Displacement/base:* operand is in memory, the address is the sum of a register and an offset in the instruction.
- *PC-relative:* operand is in memory, the address is the sum of the program counter and a constant in the instruction.

Examples:

- Load word: `lw $15,100($2)` means $15 = mem[$2 + 100]. The machine language in demcimal is [35, 2, 15, 100].
- Save word: `sw $15,100($2)` means mem[$2 + 100] = $15.
- Branch if equal: `beq $19, $20, L1` means if $19 == $20, then PC = PC + 4 + 4 * L1 (4* since the 16-bit address is a *word* offset). The machine language in decimal is [4, 19, 20, L1].
- Branch if not equal: `bne $19, $20, L1` means if $19 != $20, then PC = PC + 4 + 4 * L1.
- Addi: `addi $19, $19, 4` means $19 = $19 + 4. The machine language in decimal is [8, 19, 19, 4], where 4 is the 16-bit *immediate*.

J-format instruction:

op	address
2	10000
6 bits	26 bits

The 26-bit address is a *word* address, so it is effectively a 28-bit byte address (can address up to 256 MiB). The program counter is 32-bit, so only the lower 28 bits of the PC are replaced.

Examples:

- Jump instruction: `j exit` means jump to address with label `exit`. The machine language in decimal is [2, 10000].
- Jump and link: `jal A` means call procedure A and save the return address (PC + 4) in register $31. If a procedure calls another procedure, the return address must be stored on the stack (fills top-down in MIPS), where the address of the bottom of the stack is in register $29.

Parameter passing: The first 4 parameters are passed in registers $4 to $7, and the rest are passed on the stack. If another procedure is called, or registers are used as scratch space, registers need to be saved and restored, with two conventions:

- *Caller save:* The calling routine saves the register on the stack.
- *Callee save:* The called routine saves the register on the stack.

MIPS data sheet terms:

- *Zero-extend:* Fill the upper bits with 0s.

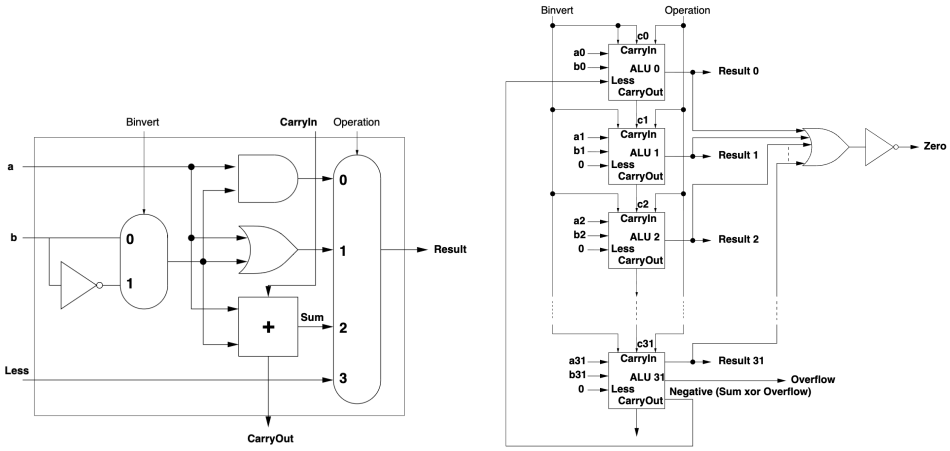
- *Sign-extend:* Take the most significant bit (the sign bit) of the smaller number and fill the upper bits with it.
- *Traps:* CPU will trigger an exception or interrupt. The address of the offending instruction is stored in a register and the computer jumps to a predefined address to invoke the appropriate handling routine.

3 ALU Design

Full adder: Simpliefied boolean expression for the Sum and CarryOut:

$$\text{CarryOut} = \text{a.CarryIn} + \text{b.CarryIn} + \text{a.b}$$
$$\text{Sum} = \text{a.}\overline{\text{b}}\text{.}\overline{\text{CarryIn}} + \overline{\text{a}}\text{.b.}\overline{\text{CarryIn}} + \overline{\text{a}}\text{.}\overline{\text{b}}\text{.CarryIn} + \text{a.b.CarryIn}$$

1-bit ALU: Can perform AND, OR, addition, subtraction, and `slt`. For subtraction, set Binvert and CarryIn of the LSB to 1. For `slt`, use the Less input.



Ripple-carry 32-bit ALU: Connect 32 1-bit ALUs, propagating the carry from one bit to the next. `slt` is performed by subtracting b from a and passing the Less signals to the Result line (Result 0 = 1 if a < b).

Carry-lookahead 16-bit ALU: Anticipate the CarryIn signal wihout waiting for it to propagate through the adder. Define:

- *Generate:* gi = ai.bi (carry generated by bit i)
- *Propagate:* pi = ai + bi (carry propagated by bit i)

4-bit carry-lookahead adder:

c1 = g0 + p0.c0

c2 = g1 + p1.c1 = g1 + p1.g0 + p1.p0.c0

c3 = g2 + p2.c2 = g2 + p2.g1 + p2.p1.g0 + p2.p1.p0.c0

c4 = g3 + p3.c3 = g3 + p3.g2 + p3.p2.g1 + p3.p2.p1.g0 + p3.p2.p1.p0.c0

Connect 4 4-bit adders using a higher level of carry-lookahead:

P0 = p0.p1.p2.p3, G0 = g3 + p3.g2 + p3.p2.g1 + p3.p2.p1.g0

P1 = p4.p5.p6.p7, G1 = g7 + p7.g6 + p7.p6.g5 + p7.p6.p5.g4

P2 = p8.p9.p10.p11, G2 = g11 + p11.g10 + p11.p10.g9 + p11.p10.p9.g8

P3 = p12.p13.p14.p15, G3 = g15 + p15.g14 + p15.p14.g13 + p15.p14.p13.g12

The carry-in signals are:

C1 = G0 + P0.c0

C2 = G1 + P1.c4 = G1 + P1.G0 + P1.P0.c0

C3 = G2 + P2.c8 = G2 + P2.G1 + P2.P1.G0 + P2.P1.P0.c0

C4 = G3 + P3.c12 = G3 + P3.G2 + P3.P2.G1 + P3.P2.P1.G0 + P3.P2.P1.P0.c0

Result is produced after 9 gate delays (each sum-of-product = 2 gate delays). For an n-bit adder, if m carrying signals are generated at each level, $\log_m n$ levels are needed.

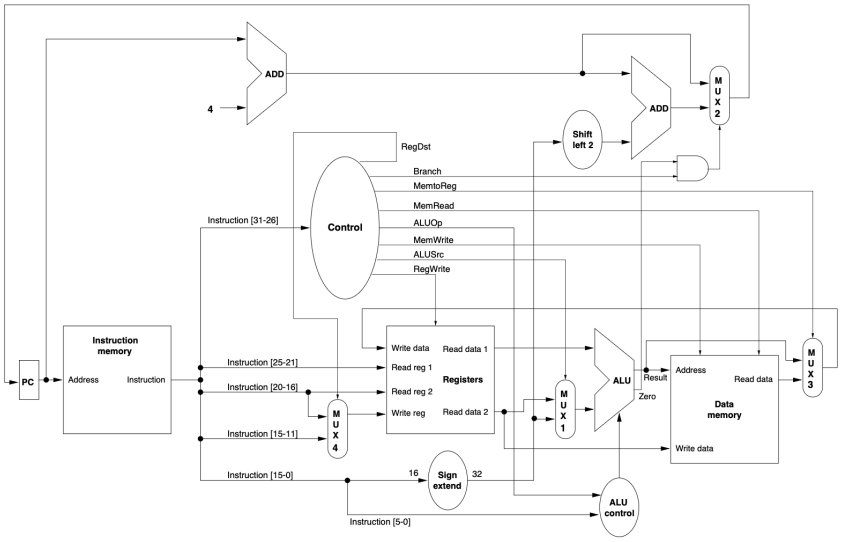
Gate delay	Signals available	Generated from
0	ai,bi,c0	—
1	gi,pi	ai,bi
3	Pi,Gi	pi,gi
5	Ci	Pi,Gi,c0
7	ci	gi,pi,Ci
9†	Sums	ai,bi,ci

† Assuming no latency for signal inversion.

	time	**space**
ripple carry	$O(n)$	$O(n)$
carry lookahead	$O(\log n)$	$O(n \log n)$
carry select	$O(\sqrt{n})$	$O(n)$

4 Datapaths and Pipelining

Datapath:



The *register file* has 3 5-bit inputs to select the 2 registers to be read and 1 register to be written. It also has 2 32-bit outputs and 1 32-bit input for reading and writing data.

The multiplexers MUX1, MUX2 and MUX3 select signals based on the instruction. Bits 26-31 of the instruction always define the operation class. For R-format instruction, bits 0-5 defines the ALU operation.

Datapath control:

Signal	Effect when low	Effect when high
MemRead	None	Data memory contents at the applied address are put on read data output
MemWrite	None	Data memory contents at the applied address replaced from write data input
ALUSrc	The second ALU operand comes from the second register file output	The second ALU operand is the sign-extended bits 0–15 of the instruction
RegDst	The write register is identified in the rt field (`lw`)	The write register is identified in the rd field (R-format)
RegWrite	None	The register identified on the write register input is replaced with the value on the write data input
Branch	The PC is replaced by the output of the adder that computes PC+4	If the zero line is high, the PC is replaced by the output of the adder which computes the branch target
MemtoReg	The value fed to the register write data input comes from the ALU	The value fed to the register write data input comes from data memory

Instruction	Format	ALUOp	Function code (inst. [5–0])	Desired ALU action	ALU control input
`lw`	I-type	00	XXXXXX	add	010
`sw`	I-type	00	XXXXXX	add	010
`beq`	I-type	01	XXXXXX	subtract	110
`add`	R-type	10	100000	add	010
`sub`	R-type	10	100010	subtract	110
`and`	R-type	10	100100	and	000
`or`	R-type	10	100101	or	001
`slt`	R-type	10	101010	set-on-less-than	111

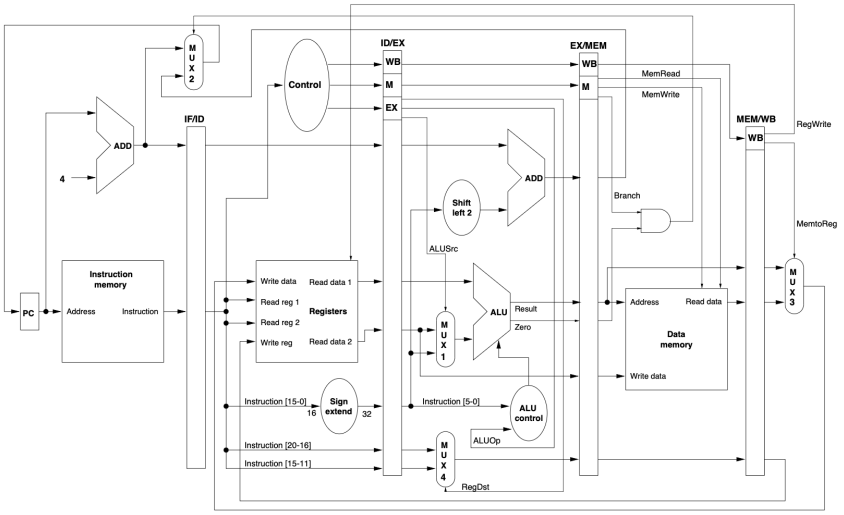
A truth table of the control signals for each instruction (bits 0-5 and 26-31) can be implemented in *ROM or Programmable Logic Array*.

Multiple clock-cycle datapath: Instructions are broken up into steps, with each step taking one cycle. The control is implemented using a *state machine* consisting of a set of states and a function to find the next state given the current state, and a set of outputs (control signal). Complex control signals can be instead implemented using a *microprogram*.

Pipelining: Split the instruction execution cycle into 5 *pipe stages*:

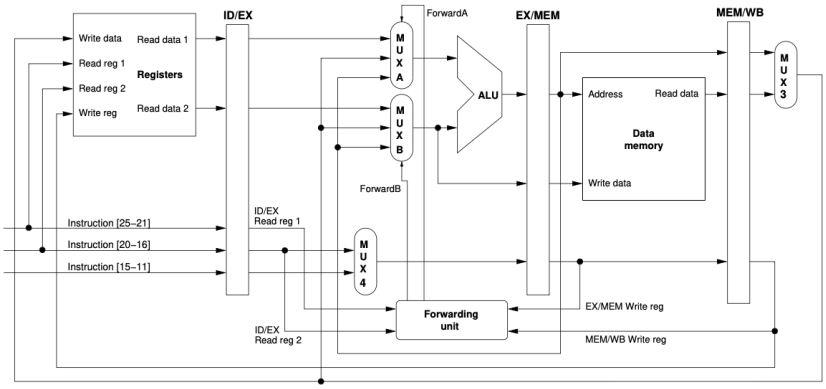
1. IF: Instruction fetch
2. ID: Instruction decode and register fetch
3. EX: Execution and effective address calculation
4. MEM: Memory access
5. WB: Write back to register file

For a large number of instructions, the speed-up approaches 4:1. Extra *pipeline registers* stores the result from one pipe stage so that it can be used by the next.



Data hazards: Data hazards occur when a value required in a register is not valid because a previous instruction has not yet updated it. Solutions include:

- *Stall:* Stages of dependent instructions are not executed until value is available. Creates *bubbles* into the pipeline and reduces the computer's efficiency. Implemented by adding a *hazard detection unit*.
- *Data forwarding:* Forward values directly to the ALU inputs, bypassing the register write-back. Implemented by adding multiplexors to the inputs of the ALU and supplying suitable control lines.



The forwarding unit compares the registers that have just been read with any registers about to be written by the two downstream instructions (at stages MEM and WB). If there is a match, MUXA and/or MUXB are set to ignore the value read from the register file and instead use the appropriate forwarded value.

Not working for `lw` as data is read from memory and required at the ALU at the same time. Can be solved by stalling or requiring the software to follow `lw` with an instruction indepedent of the load – *delayed load*.

Branch hazards: The address of the next instruction is not known until the branch instruction reaches stage 4 of the pipeline. Solutions include:

- Always stall on branches.
- Assume branch not taken and execute next instructions. If the branch is actually taken, flush the pipeline and discard the partial results, resuming execution at the branch target address.
- Predict the branch outcome.
- Delayed branches: The instruction following the branch instruction is always executed (independent of the branch outcome).

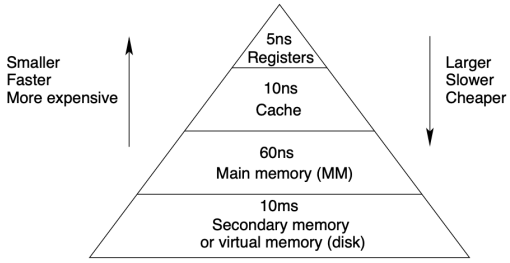
Superpipline and superscalar: Superpiplining split the pipe stages into smaller stages with shorter clock cycles. Superscalar allows multiple instructions to be executed at the same clock cycle.

5 Memory Systems

Memory hierarchy: At each level of the hierarchy, the information stored is a subset of that lower down, based on the principle of locality:

- *Temporal locality:* if an item is referenced, it tends to be referenced again soon (loops, local variables).
- *Spatial locality:* if an itme is referenced, items with nearby addresses will tend to be referenced soon (instructions, data in arrays).

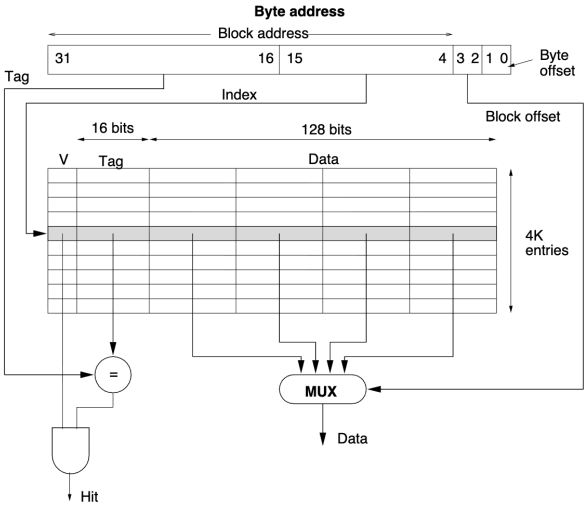
A *hit* occurs if requested data is present in an upper level, otherwise a *miss*. Aim for high hit rate and low miss penalty.



Cache: Words in a cache are arranged in *blocks*. If a word is not in the cache, the block containing the word will be loaded from main memory.

Direct-mapped cache: For every main memory address, there is a unique cache location (identified by an *index*) that can hold it.

$$\text{index} = \underbrace{\left\lfloor \frac{\text{byte address}}{\text{\# bytes per block}} \right\rfloor}_{\text{block address}} \mod \text{\# blocks in cache}$$



To reference a word in the cache:

- 12-bit *index* identifies the block in the cache.
- 16-bit *tag* identifies the block in main memory, compared with the stored tag to check for a hit. (1 cache block can correspond to 2^{16} blocks in main memory)
- 2-bit *block offset* identifies the word within the block.
- The *valid bit* is set low on start-up.

To handle a cache miss, stall the entire machine and do:

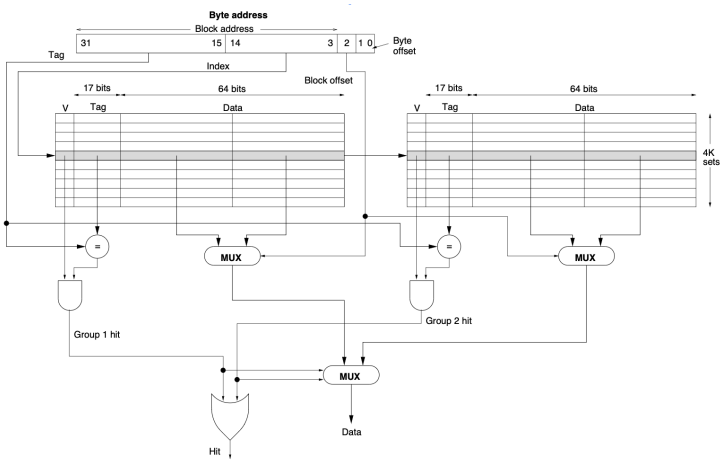
1. Reset the PC to PC - 4.
2. Instruct main memory to read the requested block and wait for completion.
3. Write the data and tag into the cache and set the valid bit to 1.
4. Restart the pipeline, which will fetch the instruction again and get a hit this time.

Cache writes: Update the main memory when the cache is updated, two approaches available:

- *Write through:* data is written to the cache and also the main memory. *Write buffers* can be used to avoid stalling.
  Advantages: read misses are cheaper since they does not require a write to main memory & easier to implement.
- *Write back:* data is written only to the cache, and the modified cache block is written to main memory only when it is replaced.
  Advantages: words are written at cache rather than main memory speed & multiple writes to a block require only one write to main memory.

Associative caches:

- *Fully associative cache:* A memory block can reside anywhere in the cache. A full search of the cache is needed to determine if a word is present.
- *Set associative cache:* A memory block can reside in a number of locations in a unique set. Only a search within the set is needed. The index of the set is (block address) mod (# sets in cache).



Block replacement: Two common strategies:

- *Random:* Randomly select a block to replace.
- *Least recently used (LRU):* Replace the block that has been unused for the longest time. Efficient at larger degrees of associativity.

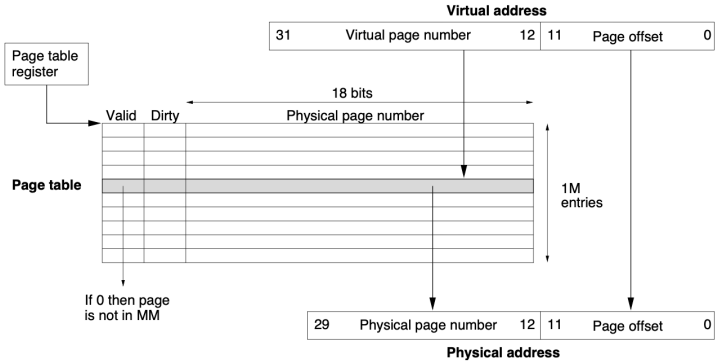
Cache performance:

- Execution time = (CPU execution cycles + memory stall cycles) $\times$ clock cycle time

- Memory stall cycles = # instructions × misses per instruction × miss penalty

Virtual memory: Virtual addresses are used and translated into physical address by the operating system. The main memory and virtual memory are divided into *pages*.

Main memory acts as a cache for virtual memory with a block size of one page. A *page fault* corresponds to a cache miss.

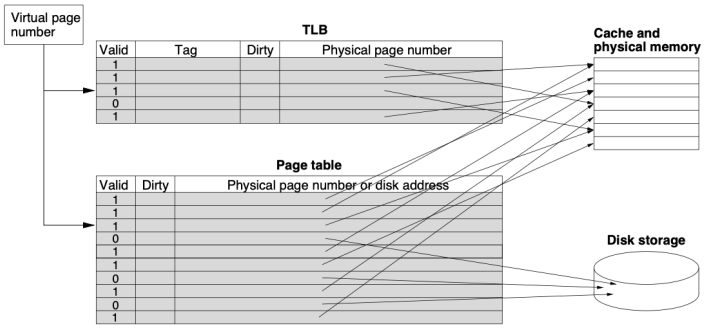


The location of pages (fully associative) is stored in a *page table* indexed by the virtual page number and contains the physical page number. Each running program has its own page table. A page table register points to the start of the page table. For each page table entry, a valid bit determines whether the page is in memory.

Write-back is used, with a *dirty bit* to indicate whether the page has been modified.

Translation lookaside buffer (TLB): A special, fast cache to store recently used translations. The TLB only caches part of the page table that point to main memory (not disk), often fully associative with random placement.

Tag = virtual page number, data = physical page number. Write back is used to ensure the dirty bits are kept up to date.



6 Interfacing to I/O Devices

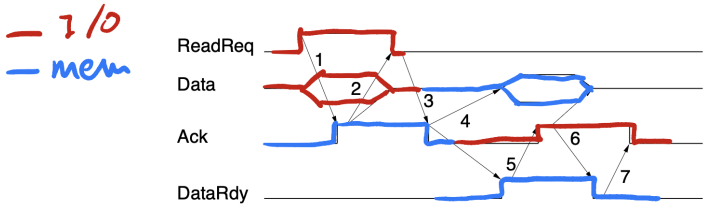
Bus: A bus is a shared communication link which uses one set of wires to connect multiple subsystems. There are two main classes of buses:

- *Processor-memory:* short, high performance, matched to memory system.
- *I/O:* long, wide range of data rates, used to connect diverse devices.

And two schemes for bus communications:

- *Synchronous:* control lines include a clock signal, all transfers are relative to the clock. Buses run at the speed of the slowest device.
- *Asynchronous:* no clock, every transfer is acknowledged by the recipient before more data is sent. Can be long since there is no clock skew.

Handshaking: Three control lines: *ReadReq* (indicate a read request), *DataRdy* (indicate that data is available), *Ack* (acknowledge the ReadReq or DataRdy).



Exchange starts when the device raises ReadReq and puts address on data lines.

1. Memory sees ReadReq, reads address, raises Ack.
2. I/O sees Ack, releases ReadReq and data lines.
3. Memory sees ReadReq low and drops Ack.
4. When data is ready, data is put on data lines and DataRdy is raised.
5. I/O sees DataRdy, reads data and raises Ack.
6. Memory sees Ack, drops DataRdy and releases data lines.
7. I/O sees DataRdy low and drops Ack.

I/O event presentation:

- *Memory-mapped I/O:* Each device appears as a set of data and control registers at a particular address. The control register contrains status bits that can be continully cheked (*polling*).
- *Interrupt-driven I/O:* An I/O device generates an interrupt when it requires attention. The CPU:
  1. Stops executing the current process.
  2. Finds the cause of the interrupt.
  3. Services the interrupt.
  4. Resumes execution of the original process.

Direct memory access (DMA): Transfer data directly between memory and I/O devices without CPU involvement (the DMA controller is the bus master instead). CPU is interrupted at the start and end of each DMA.

Steps in a DMA transfer:

1. The processor supply the DMA controller with: the identity of the device, the operation to perform, the source/destination memory address and the number of bytes to transfer.
2. The DMA controller starts the transfer of the data block (may take many transactions). After each transaction, the DMA controller updates the memory read/write address.
3. Once the DMA transfer is complete, the DMA controller interrupts the CPU, which then becomes bus master and checks that the operation completed successfully.

DMA and virtual memory:

- If the DMA controller uses physical addresses, all DMA transfer must be constrained to stay within one page.
- If the DMA controller uses virtual addresses, it needs to know the translations.

Problems with DMA in cache systems:

- If the DMA places data directly from disk into memory, the cache may contrain stale data.
- If the cache is write-back, the DMA unit might transfer stale data from memory to disk, by passing the newer data in the cache.

Solutions:

- Route all DMA transfers through the cache (expensive).
- Invalidate the cache for an I/O write or force write-backs for an I/O read (cache flushing).
- Selectively flush or invalidate individual caches entries for DMA.

Point-to-point networks: Use a serial switched point-to-point network between devices to bypass the limit on bus length.

7 Parallel Processing

Flynn's classification of parallelism:

- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data stream (SIMD)
- Multiple instruction stream, single data stream (MISD)
- Multiple instruction stream, multiple data stream (MIMD)

SIMD computers: Operate on vectors of data. SIMD instructions are broadcast to all execution units over a network, each unit has its own registers and memory. Execution units are synchronised, each handling a different data element.

SIMD machines works best on arrays and vectors.

MIMD computers: Multiple SISD machines connected on a single bus or network. Can be used to run a separate process on each processor or a parallel processing program. The processors can communicate in two ways:

- *Shared memory:* All processors can access a common memory with a single address space. Communication by loads and stores.

  If the memory takes the same time for all processors, it has a *uniform memory access (UMA)* or *symmetric multiprocessor (SMP)*. Alternative is *non-uniform memory access (NUMA)*.

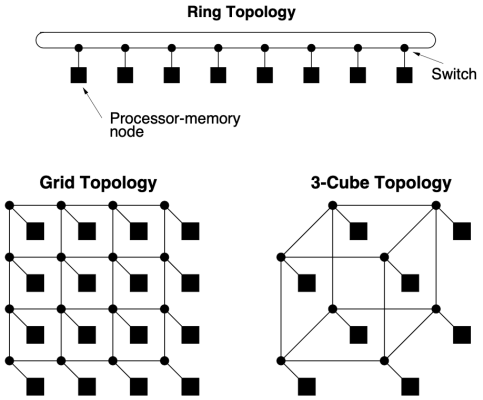
- *Message passing:* Each processor has its own private memory. Slower but synchronisation is implicit.

Snooping: All cache controllers monitor the bus and react to reads and writes in other caches to ensure cache coherence. Two ways to deal with writes:

- *Write-invalidate:* the writing processor broadcasts an invalidate signal to all other caches to invalidate the copy of the block. (practical SMP systems use write-back caches with write-invalidate since it reduces bus traffic)
- *Write-update:* the writing processor broadcasts the new data over the bus for other caches to update the copy of the block. (like a write-through cache)

On a read miss, the cache notifies other caches to check if they have a more up-to-date copy of the block.

MIMD networks: Networks allow more processors than a single bus, where each processor has a local memory (*distributed memory*). Fully connected network has link between every pair of nodes but is expensive.



Networks performance can be measured by:

- *Total network bandwidth* = # links $\times$ bandwidth per link (best possible performance).
- *Bisection bandwidth:* divide the network into two and sum the bandwidth of the links crossing the division (closer to the worst case).

Simultaneous multithreading (SMT): Run multiple processes concurrently on a superscalar processor. Separate register file, PC, and TLB are required for each process. ($\neq$ multi-core CPUs – multiple entire processors on the same chip).